

The LLM Training Core

Model Design · Scaling Laws · Training · Time & Cost

Exam-notes format, decision-grade depth. Each concept follows the same rhythm: **one-line intuition** → **key points** → **the named formula** → **a worked number** → **what to actually decide**. Formulas are stated and used, not re-derived from axioms — enough to reason and make your own calls, not to write a thesis.

Anatomy & param-count math

SwiGLU · RMSNorm · RoPE · MoE

Kaplan vs Chinchilla

5T tokens → how many params

AdamW & the 16 bytes/param rule

DP/TP/PP/ZERO

$C = 6ND$

7B → 405B cost table

The spine of the whole document, in one line: a model's *shape* sets its parameter count → scaling laws turn a *compute budget* into the right params-vs-tokens split → training memory and parallelism decide *what hardware* you need → $C = 6ND$ turns it all into *time and dollars*. Four parts, one chain.

How to read these notes for decisions. You don't need to memorize derivations. You need: (1) which knob moves which cost, (2) the one formula per topic, (3) the order-of-magnitude of the answer. Every "Decide" box distills exactly that.

Worked examples anchored to Llama-class configs and H100 (BF16) hardware. Frontier-lab specifics are public estimates. Self-contained — no external references needed to follow.

A · Model Design — how a model's shape is chosen

Why this comes first: every downstream cost (memory, FLOPs, time, dollars) is a function of the model's *shape*. Pick the shape wrong and no amount of GPUs saves you.

A1 · The dimensions you actually choose

- **N** — total parameters (the headline number, e.g. 7B). *Derived* from the others, not set directly.
- **d_{model}** — hidden size / width (e.g. 4096). The vector size flowing between blocks.
- **L** — number of transformer layers / depth (e.g. 32).
- **h** — attention heads; **d_{head}** — dim per head. Constraint: $d_{\text{model}} = h \times d_{\text{head}}$.
- **d_{ff}** — FFN inner width, usually $\approx 4 \times d_{\text{model}}$ (or $\sim 2.7 \times$ $\times 3$ matrices for SwiGLU).
- **V** — vocabulary size (e.g. 32k–128k tokens).
- **context length** — max sequence the model handles (sets KV-cache & activation cost, not param count).

Rule of thumb for "good shape": width-to-depth ratio d_{model} / L typically lands $\sim 100\text{--}150$ for dense LLMs. Too deep-and-thin trains unstably; too wide-and-shallow wastes parameters. Llama-2-7B: $4096/32 = 128$. ✓

A2 · Counting parameters from the shape

Intuition: almost all parameters live in the repeated layer stack; embeddings are a rounding error for big models. Per layer you pay for attention projections + the FFN.

PARAMETER COUNT (DECODER TRANSFORMER, ESTIMATE)

Per layer $\approx 12 \cdot d_{\text{model}}^2$ ($4 \cdot d^2$ attention QKV0 + $8 \cdot d^2$ FFN at $d_{\text{ff}}=4d$)

$N \approx 12 \cdot L \cdot d_{\text{model}}^2 + V \cdot d_{\text{model}}$ (stack + embeddings)

Worked — does Llama-2-7B's shape give 7B?

$d_{\text{model}}=4096, L=32, V=32000$

stack = $12 \times 32 \times 4096^2 = 384 \times 16,777,216 = 6.44 \times 10^9$

embed = $32000 \times 4096 \approx 0.13 \times 10^9$ ($\times 2$ if input+output untied $\approx 0.26B$)

$N \approx 6.44B + 0.26B = \sim 6.7B$ ✓ (matches the real model)

Decide: to grow a model, widening (d_{model}) costs parameters *quadratically* (d^2), deepening (L) costs them *linearly*. Want 2× params? Either 2× the layers, or 1.41× the width. Width buys more capacity per layer but costs more memory bandwidth per token.

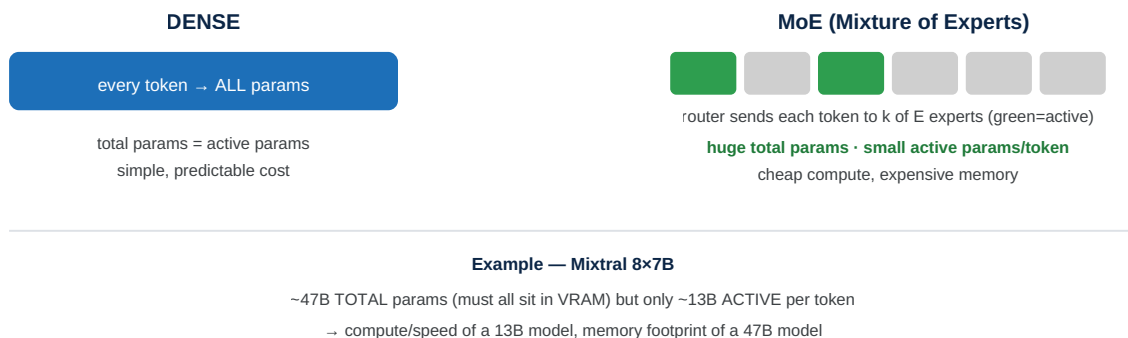
A3 · The component choices (and why the modern default won)

Slot	Old default	Modern default	Why it changed (one line)
Activation	ReLU → GELU	SwiGLU	Gated unit, better quality per param; costs a 3rd FFN matrix
Normalization	LayerNorm (post)	RMSNorm (pre)	Cheaper (no mean-centering), pre-norm trains deep nets stably
Position	Learned absolute	RoPE	Rotary; encodes relative position, extrapolates to longer context

Attention KV	MHA	GQA	Shrinks KV cache ~8× at serving with ~no quality loss
Capacity	Dense	Dense or MoE	MoE: more total params, only a fraction active per token

Exam one-liners: *RMSNorm* = LayerNorm minus the mean-subtraction (just divide by RMS, one learned scale).
RoPE = rotate Q,K by an angle proportional to position, so their dot-product depends on relative distance. *SwiGLU* = $(xW_1 \odot \text{Swish}(xW_2))W_3$ — a gate ($\odot$) controls information flow.

A4 · Dense vs MoE — the one diagram that matters



MoE decouples "how smart" (total params) from "how expensive per token" (active params). You trade VRAM for compute.

Decide Dense vs MoE: choose **MoE** when you're compute/latency-bound but VRAM-rich (serving lots of tokens cheaply). Choose **Dense** when VRAM-bound or simplicity matters. MoE's catch: you load ALL experts into memory but use few — and the router can load-imbalance, leaving some experts idle.

B · Scaling Laws — turning a compute budget into a model

Why they exist: training runs cost millions. Scaling laws let you *predict* the loss of a model you haven't trained yet, so you spend the budget on the optimal shape instead of guessing.

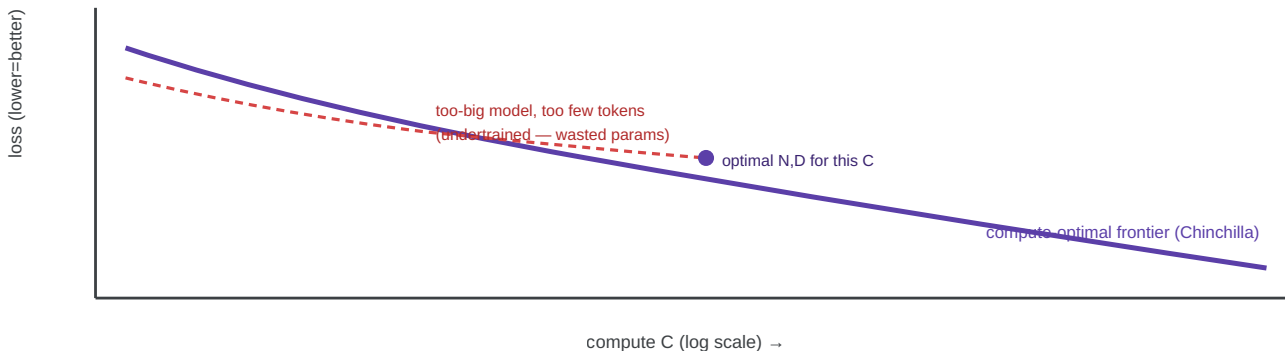
B1 · The two laws you must name

- **Kaplan et al. (2020, OpenAI)** — loss falls as a power law in params N , data D , and compute C . Their (later-corrected) takeaway: given more compute, make the **model much bigger**, grow data only slowly.
- **Chinchilla / Hoffmann et al. (2022, DeepMind)** — the correction. For a fixed compute budget, N and D should grow **together, ~equally**. Most big models of that era were *undertrained* (too big, too little data).

CHINCHILLA COMPUTE-OPTIMAL RULE OF THUMB

$D \approx 20 \times N$ (train ~20 tokens per parameter)

training compute $C \approx 6 \cdot N \cdot D$ (FLOPs; the "6" = 2 fwd + 4 bwd)



For a given compute budget there's an optimal N (and matching $D \approx 20N$). Bigger isn't better if you can't feed it enough tokens.

B2 · The exact question: 5 trillion tokens → how many parameters?

Worked — dataset-first sizing (D given)

Given $D = 5 \times 10^{12}$ tokens. Chinchilla-optimal: $N \approx D / 20$

$N \approx 5 \times 10^{12} / 20 = 2.5 \times 10^{11} = 250B$ parameters

Compute to train it: $C = 6 \cdot N \cdot D$

$= 6 \times 2.5e11 \times 5e12 = 7.5 \times 10^{24}$ FLOPs

Worked — budget-first sizing (C given)

Given a budget C . Optimal split: $N^* \propto C^{0.5}$, $D^* \propto C^{0.5}$.

Combine with $D=20N$ and $C=6ND \rightarrow C = 6 \cdot N \cdot 20N = 120 N^2$

So $N^* = \sqrt{(C / 120)}$ and then $D^* = 20 \cdot N^*$.

e.g. $C = 1e24 \rightarrow N^* = \sqrt{(1e24/120)} \approx 9.1e10 \approx 91B$ params, ~1.8T tokens

Decide — but know the modern twist (over-training): Chinchilla minimizes *training* cost. In production you ALSO pay *inference* cost forever. A smaller model trained on far more than 20× tokens is slightly suboptimal to train but

much cheaper to serve. That's why Llama-3-8B saw ~15T tokens (~1875 tokens/param, ~90× Chinchilla). **If you'll serve it a lot, deliberately under-size N and over-feed D.**

Interview trap: "Chinchilla says train on 20 tokens/param" is the *training-optimal* answer. The complete answer adds: *inference-aware* teams over-train smaller models because serving cost dominates total cost of ownership.

C · Training — the loop, the memory, the parallelism

C1 · The loop in one breath

Surface: repeat forever — push a batch forward, measure how wrong (loss), compute how each weight contributed to the wrongness (backprop), nudge weights down the gradient (optimizer step).

THE TRAINING STEP

forward → **loss** → backward (gradients) → **optimizer update** → repeat

Cross-entropy loss + perplexity

NEXT-TOKEN LOSS

$L = - (1/T) \sum_t \log p_{\theta}(x_t | x_{<t})$ (avg negative log-prob of the true next token)

Perplexity = e^L ("how many tokens the model is effectively choosing between")

Worked

Model assigns the true next token prob 0.25 on average → $L = -\ln(0.25) = 1.386$ → $PPL = e^{1.386} = 4.0$. Lower loss → lower perplexity → less "surprised."

Backprop & the optimizer (SGD → Adam → AdamW)

- **Backprop** = chain rule applied layer-by-layer to get $\partial L / \partial W$ for every weight. One backward pass $\approx 2\times$ the cost of the forward pass.
- **SGD:** $W \leftarrow W - \eta \cdot g$. Simple, but one global learning rate suits no one parameter.
- **Adam:** per-parameter adaptive steps using running mean (m) and variance (v) of gradients. Needs to *store* m and v → 2 extra copies of the parameters.
- **AdamW:** Adam with weight decay decoupled from the gradient — the LLM standard.

ADAMW UPDATE (THE STATE IS THE POINT)

$m \leftarrow \beta_1 m + (1-\beta_1)g$ (1st moment)

$v \leftarrow \beta_2 v + (1-\beta_2)g^2$ (2nd moment)

$W \leftarrow W - \eta \cdot (\hat{m} / (\sqrt{\hat{v}} + \epsilon)) - \eta \cdot \lambda \cdot W$ (adaptive step + decoupled decay)

C2 · Training memory — the 16-bytes-per-parameter rule

Why training needs ~8× the memory of inference: you must store not just weights, but gradients AND the optimizer's two moment buffers AND an fp32 master copy. This is the #1 thing people under-budget.

Per-parameter memory, mixed-precision AdamW

weights fp16-2B	grads fp16-2B	master wts fp32-4B	Adam m fp32-4B	Adam v fp32-4B	= 16 bytes / param
--------------------	------------------	-----------------------	-------------------	-------------------	--------------------

Plus ACTIVATIONS (separate, scale with batch × seq-len) — reduced by gradient checkpointing.

7B model: 7e9 × 16 = 112 GB of optimizer/model state alone — before activations.

→ won't fit one 80GB H100. This is WHY large-model training is distributed (see C4).

Inference needs ~2 bytes/param (weights only). Training needs ~16. That 8× gap is the entire reason for ZeRO/FSDP.

TRAINING STATE MEMORY

$M_{\text{state}} \approx 16 \cdot N \text{ bytes}$ (fp16 wts+grads + fp32 master+m+v)

$M_{\text{total}} = M_{\text{state}} + M_{\text{activations}}$

Worked — can a 70B model train on 8×H100 (640 GB)?

$M_{\text{state}} = 70e9 \times 16 = 1,120 \text{ GB}$. Plus activations.

640 GB available < 1,120 GB needed → **No, not without sharding.**

With ZeRO-3 / FSDP splitting state across all 8 GPUs: $1,120 / 8 = 140 \text{ GB/GPU}$ — still > 80.

→ need ≥ ~24 H100s ($1120/80 \approx 14$, +activations+overhead) just to *hold* the state. **Real 70B runs use 256–1024 GPUs** for speed, not just fit.

Decide: gradient **checkpointing** trades compute for memory — recompute activations in the backward pass instead of storing them. Cuts activation memory dramatically (~√L) at ~30% extra compute. Turn it on when activation memory, not state memory, is your wall.

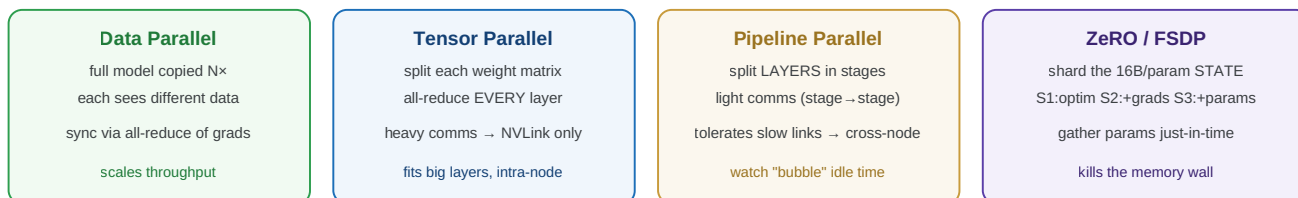
C3 · Stabilizers & schedules (the knobs that prevent divergence)

Knob	One-line job	Typical value
Warmup	Ramp LR from ~0 over first steps so early huge gradients don't blow up	0.5–2% of steps
Cosine decay	Smoothly lower LR to near-0 by the end for fine convergence	decay to ~10% of peak
Weight decay (λ)	Pull weights toward 0; regularizes	~0.1
Grad clipping	Cap gradient norm so a bad batch can't spike the update	max-norm 1.0
Global batch	Tokens per optimizer step; big = stable but needs LR tuning	1M–16M tokens

Micro-batch → global batch: GPU can't fit the huge global batch at once, so you run small **micro-batches**, **accumulate** their gradients over several steps, then do one optimizer update. $\text{global} = \text{micro} \times \text{accumulation} \times \text{data-parallel-replicas}$.

C4 · Parallelism — the 4 ways to split a model (+ ZeRO)

Surface: one model won't fit/train fast on one GPU, so you split it. Four axes, combined as needed; total GPUs ≈ DP × TP × PP.



Real frontier recipe = combine them: e.g. TP=8 (within node) × PP=8 (across nodes) × DP=16

= 1024 GPUs, with ZeRO/FSDP sharding the optimizer state on top. This is "3D parallelism."

Why GPUs synchronize: each holds partial results; all-reduce sums+shares them so every GPU agrees.

NCCL = NVIDIA's library doing this over NVLink/InfiniBand. Ring all-reduce moves $\approx 2 \times (\text{model size})$ of data per step.

Communication, not compute, often becomes the bottleneck at scale — why interconnect (NVLink/IB) matters as much as FLOPs.

Four orthogonal splits. DP scales speed; TP/PP/ZeRO defeat the memory wall. The art is balancing them against your interconnect.

Decide which parallelism: start with **ZeRO/FSDP + DP** (simplest, handles most $\leq 70\text{B}$ cases). Add **TP** when a single layer's activations/weights overflow one GPU (needs NVLink). Add **PP** only when you must span many nodes and TP's comms saturate. Rule: *TP within a node, PP across nodes, DP/ZeRO on top.*

Interview Q: "Why can't you just use tensor parallelism across 100 nodes?" → TP all-reduces every layer; across slow inter-node links that communication dominates and GPUs starve. TP needs NVLink-class bandwidth, so it's capped at intra-node (~8 GPUs); scale further with PP/DP.

D · Training Time & Cost — from FLOPs to dollars

Surface: three numbers give you everything — total FLOPs ($C=6ND$), how fast your fleet does FLOPs (GPUs $\times$ peak $\times$ utilization), and price per GPU-hour.

THE THREE FORMULAS

1. Compute $C = 6 \cdot N \cdot D$ FLOPs
2. Time $t = C / (n_{\text{GPU}} \cdot F_{\text{peak}} \cdot \text{MFU})$
3. Cost $\$ = n_{\text{GPU}} \cdot t_{\text{hours}} \cdot \text{price/GPU-hr}$

- F_{peak} (H100, BF16) $\approx 1 \times 10^{15}$ FLOP/s (~ 989 TFLOPS).
- MFU = Model FLOPs Utilization = fraction of peak actually achieved. Real large runs: **0.35–0.5**. Use 0.4.
- $\text{price} \approx \$2/\text{H100-hour}$ (cloud, order-of-magnitude).

Worked end-to-end — 70B, Chinchilla-optimal ($D=20N=1.4\text{T}$)

$C = 6 \times 70\text{e9} \times 1.4\text{e12} = \mathbf{5.88 \times 10^{23} \text{ FLOPs}}$

fleet rate (1024 H100, MFU 0.4) $= 1024 \times 1\text{e15} \times 0.4 = 4.1 \times 10^{17}$ FLOP/s

$t = 5.88\text{e23} / 4.1\text{e17} = 1.43\text{e6 s} = \mathbf{\sim 16.6 \text{ days}}$

GPU-hours $= 1024 \times (16.6 \times 24) \approx \mathbf{408,000 \text{ GPU-hr}}$

cost $= 408,000 \times \$2 \approx \mathbf{\sim \$816,000}$

D1 · The reference table (Chinchilla-optimal, H100 @ MFU 0.4, \$2/hr)

Model	Tokens ($20N$)	FLOPs ($C=120N^2$)	H100-hours	$\approx$ Cost
7B	140B	5.9e21	4,100	\$8K
13B	260B	2.0e22	14,100	\$28K
70B	1.4T	5.9e23	408,000	\$816K
175B	3.5T	3.7e24	2.55M	\$5.1M
405B	8.1T	2.0e25	13.7M	\$27M

Decide / reality-check: these are *training-optimal* floors. Real frontier models train past Chinchilla (Llama-3-405B used $\sim 15\text{T}$ tokens, not 8.1T) $\rightarrow$ roughly $2\times$ these FLOPs, hours and dollars. Add $\sim 20\text{--}40\%$ for failed runs, restarts, evals, and data ablations. So a "\$27M" 405B is realistically **\$50–70M+** of compute. The table tells you the *order of magnitude* — which is what you need to decide go/no-go.

Interview math you should be able to do cold: "Roughly what does it cost to train a 7B Chinchilla-optimal model?"
 $\rightarrow C=120 \times (7\text{e9})^2 \approx 6\text{e21}; \div (1\text{e15} \times 0.4 \times 3600) = \text{GPU-hrs} \approx 4\text{k}; \times \$2 \approx \mathbf{\$8K}$. The whole chain in 20 seconds.

★ One-page exam cheat sheet

The 5 formulas

$$N \approx 12 \cdot L \cdot d_{\text{model}}^2 + V \cdot d_{\text{model}} \quad (\text{params from shape})$$
$$D \approx 20 \cdot N \quad (\text{Chinchilla tokens/param})$$
$$M_{\text{train}} \approx 16 \cdot N \text{ bytes} \quad (\text{AdamW mixed-precision state})$$
$$C = 6 \cdot N \cdot D \quad (\text{training FLOPs})$$
$$t = C / (n_{\text{GPU}} \cdot F_{\text{peak}} \cdot \text{MFU}) \quad (\text{wall-clock})$$

The decision reflexes

Question	Reflex answer
Grow params cheaply?	Add layers (linear) before width (quadratic)
How many tokens?	$\geq 20 \times N$ to train-optimal; far more if you'll serve it heavily
Will it fit for training?	$16 \times N$ bytes of state; shard with ZeRO/FSDP; checkpoint activations
Which parallelism?	ZeRO+DP default $\rightarrow$ TP within node $\rightarrow$ PP across nodes
Dense or MoE?	MoE if compute-bound & VRAM-rich; Dense if VRAM-bound
What's it cost?	$C = 120N^2 \rightarrow \div (1e15 \cdot 0.4 \cdot 3600) \rightarrow \times \2 ; then $\times 2$ for real over-training

The names to drop (and what they fix)

SwiGLU activation · RMSNorm norm · RoPE position · GQA KV-cache · AdamW optimizer · cosine+warmup LR · grad clipping/ checkpointing stability/memory · Chinchilla token sizing · ZeRO/FSDP state sharding · 3D parallelism (DP×TP×PP) · NCCL/all-reduce GPU sync · MFU efficiency.

Scope: Training-core only (Parts 4, 8, 9 of your outline) at decision-grade notes depth. Deliberately out of scope here and natural next notes if you want them: data pipelines & tokenization (Parts 2–3), post-training SFT/RLHF/DPO/LoRA (Parts 10–11), and quantization/ serving (covered in your separate KV-Cache & Inference deep dive). Numbers are order-of-magnitude planning figures; configs reflect public Llama-class models.